
APACHE AIRFLOW 3.1.7



APPENDIX B

Exercise Checklist

Danh sách bài tập thực hành cho từng Stage

Target:	Backend / Distributed Systems Engineer
Version:	Apache Airflow 3.1.7
Type:	Practical checklist
Use:	Theo dõi tiến độ học tập

Mục lục

Stage 1: Foundation Exercises

Stage 2: Architecture Exercises

Stage 3: Integration Exercises

Stage 4: Production Exercises

Stage 5: Advanced Exercises

Stage 6: Comparison Exercises

Stage 1: Foundation Exercises

Mục tiêu: Nắm vững khái niệm DAG, Operator, TaskFlow API, Docker setup.

- [] Chạy `docker compose up` và truy cập UI ở localhost:8080
- [] Viết DAG với 3 tasks: extract → transform → load (BashOperator)
- [] Convert DAG sang TaskFlow API (@dag, @task)
- [] Thử `airflow dags test stage1_invoice_validation 2026-06-01` (CLI test)
- [] Thêm `retries=2` và `retry_delay=timedelta(minutes=1)`
- [] Thử `catchup=True` với start_date cũ → observe backfill → đổi về `False`

Bảng 1: Kết quả Stage 1

Tiêu chí đánh giá	Hoàn thành	Ghi chú
Chạy Airflow tên Docker thành công	[]	
Viết được DAG cơ bản	[]	
TaskFlow API chạy đúng	[]	
Test DAG qua CLI	[]	
Hiểu rõ catchup, retries	[]	

Stage 2: Architecture Exercises

Mục tiêu: Hiểu sâu scheduler, executor, XCom, sensors.

- [] Query bảng `dag_run` và `task_instance` trong Postgres
- [] Push/pull XCom giữa 2 tasks, verify trong UI Admin → XComs
- [] Thử cả 3 sensor modes: poke, reschedule, deferrable
- [] Chạy `airflow triggerer` và test deferrable sensor
- [] Đo `dag_processing.total_parse_time` qua UI metrics
- [] Thử đổi Executor từ Local → Sequential → observe difference

Bảng 2: Kết quả Stage 2

Tiêu chí đánh giá	Hoàn thành	Ghi chú
Hiểu metadata schema	☐	
Thành thạo XCom push/pull	☐	
Test 3 sensor modes	☐	
Cấu hình triggerer	☐	
Monitor scheduler metrics	☐	

Stage 3: Integration Exercises

Mục tiêu: Tích hợp Kafka, Spark, Cassandra, REST API, Custom Operator.

- ☐ Tạo Custom Operator `XMLValidationOperator` và dùng trong DAG
- ☐ Setup Airflow Connection cho Cassandra, test hook
- ☐ Viết DAG gọi `SparkSubmitOperator` (dùng Spark local mode)
- ☐ Tạo DAG gọi REST API với httpx + mTLS (self-signed cert)
- ☐ Tạo Plugin với custom Hook cho Kafka AdminClient
- ☐ Test `airflow tasks test <dag_id> <task_id> <date>` cho từng task

Bảng 3: Kết quả Stage 3

Tiêu chí đánh giá	Hoàn thành	Ghi chú
Viết custom operator	☐	
Quản lý connections	☐	
Tích hợp Spark	☐	
API + security	☐	
Viết custom hook	☐	

Stage 4: Production Exercises

Mục tiêu: Triển khai production với Celery, monitoring, scaling, SLA.

- [] Chuyển LocalExecutor → CeleryExecutor + Redis
- [] Scale worker: `docker compose up --scale airflow-worker=3`
- [] Truy cập Flower UI (localhost:5555) và observe task queue
- [] Setup Prometheus scraping endpoint, tạo 1 Grafana panel
- [] Đặt SLA tên task và simulate miss → check email/alert
- [] Simulate worker crash: `docker kill <worker>` → observe retry
- [] Test DB connection pool: monitor `pg_stat_activity`

Bảng 4: Kết quả Stage 4

Tiêu chí đánh giá	Hoàn thành	Ghi chú
CeleryExecutor chạy ổn định	[]	
Scale worker thành công	[]	
Monitor bằng Flower	[]	
Prometheus + Grafana hoạt động	[]	
SLA alerting hoạt động	[]	
Worker crash được xử lý	[]	

Stage 5: Advanced Exercises

Mục tiêu: Asset triggers, dynamic mapping, Task SDK, RAG pipeline.

- [] Tạo Asset trigger DAG: DAG A update asset → DAG B tự động chạy
- [] Viết dynamic task mapping: 5 files → 5 parallel tasks → aggregate
- [] Thử Airflow 3.0 Task SDK: cài `pip install apache-airflow-task-sdk`, viết remote task
- [] Tạo RAG pipeline: extract PDF → chunk → embed (mock) → index

- [] Test airflow dags reserialize để verify DAG versioning
- [] Thử airflow db migrate với Alembic revision check

Bảng 5: Kết quả Stage 5

Tiêu chí đánh giá	Hoàn thành	Ghi chú
Asset trigger hoạt động	[]	
Dynamic mapping chạy đúng	[]	
Task SDK remote task	[]	
RAG pipeline hoàn chỉnh	[]	

Stage 6: Comparison Exercises

Mục tiêu: Đánh giá khách quan Airflow vs Prefect vs Dagster.

- [] Cài đặt Prefect tên local, viết 1 flow tương đương DAG Stage 1
- [] So sánh thời gian setup: Airflow vs Prefect vs Dagster
- [] Viết 1 DAG có data lineage rõ ràng, so sánh với Dagster
- [] Tính TCO (Total Cost of Ownership) cho 3 tool trong 1 năm
- [] Viết document quyết định chọn tool cho team

Bảng 6: Kết quả Stage 6

Tiêu chí đánh giá	Hoàn thành	Ghi chú
Đánh giá Prefect	[]	
Đánh giá Dagster	[]	
Phân tích chi phí TCO	[]	
Document quyết định	[]	

Mẹo hoàn thành khóa học

- Dành **2-3 giờ mỗi ngày**, không nên học 1 lúc quá nhiều
- Mỗi Stage: đọc lý thuyết trước, sau đó làm bài tập ngay
- Nếu gặp lỗi, đọc Appendix A trước khi tìm trên Google
- Ghi chép lại những gì học được — viết blog hoặc notes
- Tham gia cộng đồng: Apache Airflow Slack, Reddit r/dataengineering